

LAB # 14

Switch Control Structure in MIPS Assembly Language

OBJECTIVE

To implement translation of a “switch” control structure in mips assembly language.

Theory

`.align n`

Align next data item on specified byte boundary (0=byte, 1=half, 2=word, 3=double)

`blez $v0, label`

Branch if less than or equal to zero: Branch to statement at label's address if \$t1 is less than or equal to zero

`bgt $v0,$t3, label`

Branch if Greater Than: Branch to statement at label if \$t1 is greater than \$t2

`lw $t2,0($t1)`

Load Word : Set \$t2 to contents of effective memory word address

`jr $t2`

jr \$t2 Jump register unconditionally : Jump to statement whose address is in \$t2

`b label`

Branch : Branch to statement at label unconditionally

PROGRAM#1

Write a program in MIPS assembly language that translate SWITCH control structure.

```
***** SWITCH TRANSLATION *****
```

```
#####DATA SEGMENT #####
```

```
##### DATA SEGMENT #####
```

```
.data
```

```
input: .asciiz "\nType a value from 1 to 3: "
```

```
msg1: .asciiz "\nYou are in case 1"
```

```
msg2: .asciiz "\nYou are in case 2"
```

```
msg3: .asciiz "\nYou are in case 3"
```

```
##### TEXT SEGMENT #####
```

```
.text
```

```
.globl main
```

```
main:
```

```
    # Print input message
```

```
    li $v0, 4
```

```
    la $a0, input
```

```
    syscall
```

```
    # Read integer
```

```
    li $v0, 5
```

```
    syscall
```

```
    # If input <= 0 → repeat
```

```
    blez $v0, main
```

```
    # If input == 1 → case1
```

```
    li $t0, 1
```

```
    beq $v0, $t0, case1
```

```
    # If input == 2 → case2
```

```
    li $t0, 2
```

```
    beq $v0, $t0, case2
```

```
    # If input == 3 → case3
```

```
    li $t0, 3
```

```
    beq $v0, $t0, case3
```

```
    # Default (greater than 3)
```

```
    b main
```

```
# ----- CASE 1 -----
```

```
case1:
```

```
    li $v0, 4
```

```

    la $a0, msg1
    syscall
b end

# ----- CASE 2 -----
case2:
    li $v0, 4
    la $a0, msg2
    syscall
b end

# ----- CASE 3 -----
case3:
    li $v0, 4
    la $a0, msg3
    syscall

# ----- END -----
end:
    li $v0, 10
    syscall

```

TASK:

1. Write a program in MIPS to translate switch control structure using arithmetic calculator
2. Design a simple command line calculator which can perform basic arithmetic operations, inverse, squares and percent calculations two integers provided as input along with respective symbol

LAB # 15

PROCEDURES IN MIPS

OBJECTIVE

To implement procedure in mips assembly language.

Theory

INTRODUCTION

So far, we've only looked at programs consisting of a single long chunk of code. Each program has started at the top of the code, executed each instruction in turn (with an occasional detour for looping or decisionmaking), and then ended at the bottom of the code. That's fine for small programs, but larger programs require a programming construct known as a subroutine/procedure.

You've probably familiar with subroutines from a high-level language. In C, subroutines are known as functions, and in Pascal and Basic, they're known as procedures and functions. Subroutines, procedures, and functions all amount to the same thing – a separate section of code that optionally accepts well-defined inputs, promptly performs a certain action, and optionally returns a specific result value.

Subroutines let you build programs in a modular fashion, with the subroutines hiding the details of specific tasks so you can focus on the overall flow of the program. Subroutines can also make programs far more compact, since a single subroutine can be called from many places in a program, and can even perform different functions when passed different values. In a large program (whether written in assembler, C, Pascal, or some other language), subroutines are essential to creating orderly, maintainable code.

HOW PROCEDURES WORK

There are six steps that need to be accomplished in order to call and return from a procedure.

1. Place parameters in a place where the procedure can access them.
2. Transfer control to the procedure
3. Acquire the storage resources needed for the procedure.
4. Execute the procedure
5. Place the result value in a place where the calling program can access it.
6. Return control to the point of origin.

MIPS software follows the following convention in allocating its 32 registers for procedure calling:

- \$a0 – \$a3: four argument registers in which to pass parameters

- \$v0 – \$v1: two value registers in which to return values
- \$ra: one return address register to return to the point of origin

The code that calls the procedure executes a jal instruction, which saves the address of the following instruction (PC+4) in register \$ra and then loads PC with the address of the desired subroutine, thereby branching to the subroutine. The subroutine then executes just as any other code would. Procedures can – and often do – contain calls to other procedures; in fact, properly designed subroutines can even call themselves, a practice known as recursion.

When the subroutine has finished its task, it executes a jr \$ra instruction, which jumps to the address stored in register \$ra. This causes execution of the calling routine to resume at the instruction following the jal instruction.

However, since the procedure may utilize any registers needed by the caller, those registers must be preserved before the procedure called and then be restored back after the procedure completed the tasks. The ideal data structure for spilling registers is a stack. MIPS software allocates \$sp to track as the top of stack (TOS). The stack grows from higher address to lower address.

NESTED PROCEDURES

Procedures that do not call others are called leaf procedures. Non-leaf procedures must push all necessary registers before calling other procedures.

INSTRUCTIONS USED:

jal label

(Jump and link)

Copies the address of the next instruction into the register \$ra(register 31) and then jumps to the address label.

jr \$ra

Jump register unconditionally : Jump to statement whose address is in \$ra

lw \$t2,0(\$t1)

Load Word : Set \$t2 to contents of effective memory word address

j label

Jump Unconditrionally: jump to statement at label unconditionally

```
blez $v0, label
```

Branch if less than or equal to zero: Branch to statement at label's address if \$t1 is less than or equal to zero

PROGRAM#1

Write a program in MIPS assembly language that display sum of positive and negative values using procedure.

```
##### DATA SEGMENT #####
```

```
.data
```

```
array: .word -4, 5, 8, -1
```

```
msg1: .asciiz "\nSum of positive values = "
```

```
msg2: .asciiz "\nSum of negative values = "
```

```
##### TEXT SEGMENT #####
```

```
.text
```

```
.globl main
```

```
main:
```

```
    # Print message for positive sum
```

```
    li $v0, 4
```

```
    la $a0, msg1
```

```
    syscall
```

```
    # Pass parameters to procedure
```

```
    la $a0, array    # address of array
```

```
    li $a1, 4        # number of elements
```

```
    jal Sum          # call procedure
```

```
    # Print positive sum (returned in $v0)
```

```
    move $a0, $v0
```

```
    li $v0, 1
```

```
    syscall
```

```
    # Print message for negative sum
```

```
    li $v0, 4
```

```
    la $a0, msg2
```

```
    syscall
```

```
# Print negative sum (returned in $v1)
move $a0, $v1
li $v0, 1
syscall
```

```
# Exit program
li $v0, 10
syscall
```

```
#####
# PROCEDURE: Sum
# $a0 = address of array
# $a1 = number of elements
# $v0 = sum of positive values (return)
# $v1 = sum of negative values (return)
#####
```

Sum:

```
# Initialize sums
move $v0, $zero # positive sum = 0
move $v1, $zero # negative sum = 0
```

Loop:

```
blez $a1, Return # if no elements left, return
```

```
lw  $t0, 0($a0) # load array element
addi $a0, $a0, 4 # move to next element
addi $a1, $a1, -1
```

```
blez $t0, Negative
```

```
# Positive value
add $v0, $v0, $t0
j Loop
```

Negative:

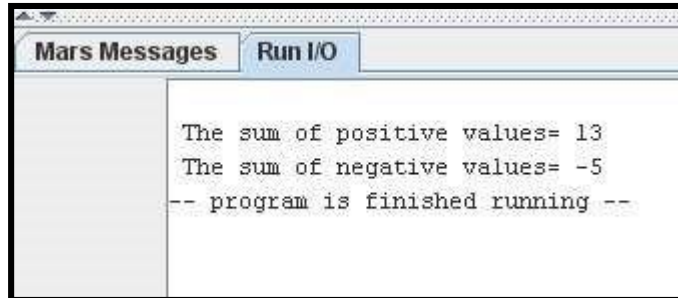
```
# Negative value
add $v1, $v1, $t0
```

j Loop

Return:

jr \$ra

OUTPUT:

A screenshot of a window titled "Mars Messages" with a "Run I/O" button. The window contains the following text:

```
The sum of positive values= 13  
The sum of negative values= -5  
-- program is finished running --
```

TASK:

1. Write a program, in MIPS that display the factorial of a number using procedures
2. Write a MIPS assembly program to generate Fibonacci sequence up to number limit input by the user